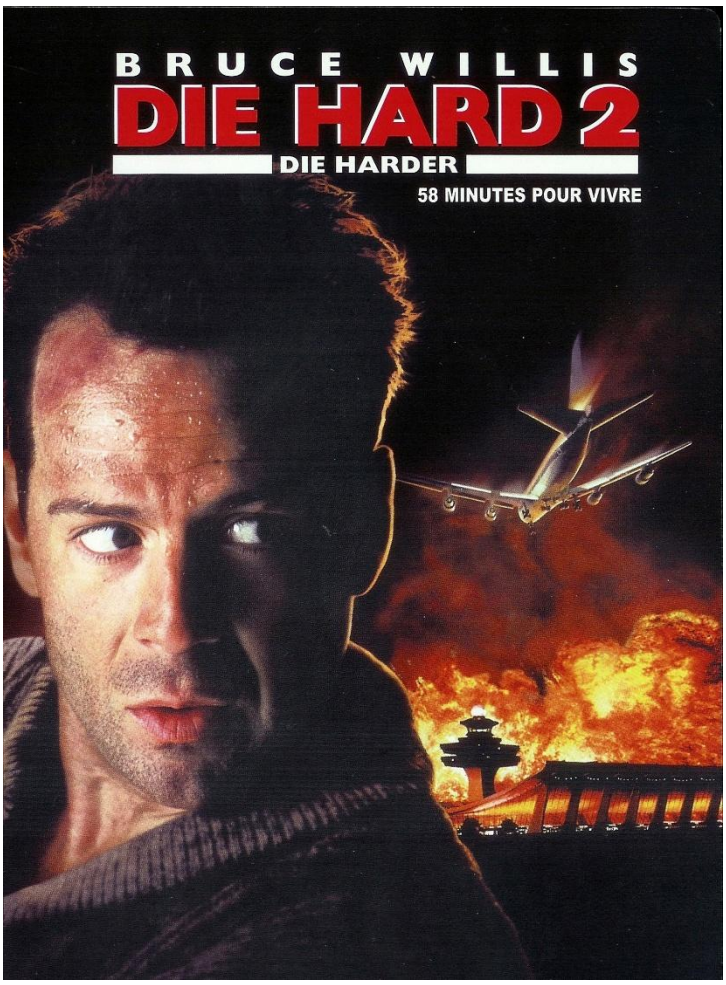
58 Minutes to Live

A Problem-Based Learning (PBL) project for students in the Advanced Algorithms 1 module



Henri HAGBE

Context

Roissy Airport is under the threat of an exceptional storm. At 19:42, two runways have just been closed and inbound traffic is held in holding patterns. The regulation center must make fast, defensible decisions: every minute counts.

- Establish a priority landing order based on heterogeneous information (remaining fuel, medical emergency, technical issue, diplomatic importance, scheduled arrival time).

- Design a decision system able to adapt: policy (POLICY) changes depending on the context, stress tests, and a response to an unforeseen event during the debrief session.

You have been commissioned as a software engineering team: your objective is to deliver a reliable, modular, defensible Python module that can simulate the impact of different choices in a critical situation. Here are the standard stress tests your application must be able to handle.

It is 19:42.

An exceptional storm is hitting the Paris region.

Two runways are closed. Only one runway remains operational.

24 aircraft are inbound.

Some are low on fuel.

Some report technical issues.

Some carry passengers in medical distress.

Some have high diplomatic importance.

The control center assigns you a mission:

Design a software module capable of determining the landing order of aircraft.

You have 58 minutes before the situation becomes critical.

Work required

You are a team of algorithmics engineers in charge of a prioritization module for air traffic control.
Your mission: design and develop in Python a system able to rank arriving aircraft according to a POLICY (admission policy), then simulate the consequences in a crisis situation (fuel, incidents, time constraints).

More specifically, your work must meet the following requirements:

1. **Data representation and traffic loading**:
    - Load the provided initial dataset (list of dictionaries representing aircraft) and verify its consistency (expected fields, values, absence of errors).
    - Implement utility functions (display, minimum search, subset extraction) to facilitate debugging and validation.

2. **Prioritization POLICIES (decision rules)**:
    - Define at least three different POLICIES (e.g., fuel priority, incident priority, diplomatic priority), as comparators or scoring functions.
    - Allow switching POLICY without rewriting the sorting algorithm (POLICY / ALGORITHM separation).

3. **Implementation and comparison of sorting algorithms**:
    - Implement at least two quadratic-time sorts (insertion sort and selection sort) to establish the landing order.
    - Empirically compare the algorithms (number of comparisons, optionally runtime) on several traffic sizes.

4. **Stress tests (robustness testing)**:
    - Program a traffic generator to create different scenarios (normal, fuel crisis, medical crisis, diplomatic summit).
    - Test your module on increasing volumes (10, 30, 50, 100 aircraft) and analyze the limits of quadratic sorting.

5. **Dynamic simulation and reporting**:

    - Simulate the passage of time: each round, one plane lands, the fuel of the others decreases, and a crash occurs if fuel ≤ 0!
    -

- Produce a summary (planes saved / crashed) and justify your choices. **In the return session, an unforeseen constraint will be added: your system will have to adapt quickly. Those who try to cheat will be out of luck**.

Data definition

The dataset describes a set of inbound aircraft and the minimum information needed to prioritize landings.

1. Identification and context information
   - **id**: Flight identifier (e.g., "AF342").
   - **arrival_time**: Scheduled arrival time (useful as a fairness criterion / tie-break).
   - **diplomatic_level**: Level of diplomatic importance (1 to 5).
   - **(Optional) airline / origin**: Descriptive information if you want to enrich the simulation.
   - **(Optional) notes**: Free field to annotate an event (e.g., "head of state", "medical cargo").

2. Safety and criticality data
   - **fuel**: Minutes of fuel remaining.
   - **medical**: Presence of an onboard medical emergency (boolean).
   - **technical_issue**: Reported technical incident / failure (boolean).
   - **(Optional) fuel_burn_rate**: Additional consumption (e.g., leak) if you model deterioration.
   - **(Optional) must_land**: Absolute priority (used during the unforeseen event).
   - **(Optional) diverted**: Marker if a flight is diverted instead of landing.

Definition of POLICIES and prioritization

A POLICY defines the decision rule: it turns heterogeneous information into a landing order.

In this project, you must separate the POLICY (what to prioritize) from the sorting algorithm (how to rank).

Recommended convention: a comparator takes two aircraft a1 and a2 and returns True if a1 must be placed before a2.

```
def policy(a1, a2):
  # True if a1 must be before a2
  ...
```

Example crisis hierarchy (adapt as needed):

- Technical incident (technical_issue) has priority
- Medical emergency (medical) has priority
- Lowest fuel (fuel) has priority
- Diplomatic importance (diplomatic_level) to break ties

In case of ties, you must define a tie-break (e.g., arrival time) and discuss the stability of your sort.

The code must be structured in a clear, modular way, following Python programming best practices:

1. **Variable clarity**: Use explicit, meaningful names for variables and functions (policy_fuel, selection_sort, simulation, ...).
2. **Code organization:** Split the code into well-defined functions (traffic generation, sorting, policies, simulation, analysis).
3. **Documentation**: Each function must include a docstring explaining its role, parameters, and return values.
4. **Project structure**: Organize the project into multiple files if needed (e.g., policies.py, sorts.py, simulation.py, main.py).
5. **Use of comments**: Add relevant comments, notably to explain priority choices (the "political" dimension of the policy).

Resources to address the problem situation

- The course/lab "Sorting algorithms 1" (selection sort, insertion sort, stability, complexity).
- The course "Python language" (lists, dictionaries, functions, modules).
- Optional: basics of performance measurement (time/perf_counter) and random generation (random module) for stress testing.
- The initial dataset (provided) and the debrief-session instructions (unforeseen event).

Expected deliverables

- The project meeting the requirements above (Python script or notebook), including: policies, quadratic sorts, stress tests and simulation.
- A test suite (or demo script) to reproduce your results across several scenarios.
- A short report (2 to 4 pages) explaining:
    - The project structure and the distribution of work within the team (roles, contributions).
    - Technical choices: aircraft representation, policies, sorting algorithms, metrics (comparisons, runtime).
    - Difficulties encountered, how you resolved them, and a critical analysis (limits of quadratic sorting, stability).

An oral presentation (5 to 7 minutes) during the debrief session: demo, policy justification, and adaptation to the unforeseen event.
Particular attention will be paid to best practices and modularity (POLICY / ALGORITHM).

Learning objectives of the PBL

- Manipulate lists of dictionaries in Python (searching, filtering, extracting, updating).
- Understand and master quadratic sorts (selection sort, insertion sort) and the notion of stability.
- Test and empirically compare algorithms (comparisons, runtime) and relate results to $O(n^2)$ complexity.
- Collaborate effectively as a team: roles, organization, optional versioning, and clear reporting.
- Design a modular decision system: separation between POLICY (rules) / ALGORITHM (sorting) / SIMULATION (dynamics).
- Develop critical analysis skills: trade-offs between “fairness / safety / efficiency” and adaptability to an unforeseen constraint.
- Follow Python best practices (naming, docstrings, structure, readability) to produce maintainable code.